

test_02_constrained.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/host_if/tests_aop/test_02_constrained.vr

```
/*
*
* Author:      Fabian Delguste
* File:        $RCSfile: test_02_constrained.vr,v $
* Revision:    $Revision: 1.1 $
* Date:        $Date: 2003/11/12 18:43:07 $
*
*****
*
* This test shows how to create constrained test by defining
* a set of scenarios
*
* The scenario implemented here performs the following
* transactions:
*   - Generates 32 WRITE cycles at address [0..31]
*     with data 0x00, 0xFF, 0x55 or 0xAA
*   - Generate N WRITE cycles at the read-only addresses [0..7]
*   - Generates 32 READ cycles at address [0..31]
*
*****
*/

#include "dut_env.vrh"

// Define a constrained scenario
extends constrained_intel_mgmt_scenario(intel_mgmt_scenario) {
    static bit [31:0] WRITES    = 1;
    static bit [31:0] WRITE_RO = 2;
    static bit [31:0] READS     = 3;

    //This constraint will replace the default 'valid_scenario'
    //'valid_scenario' is disabled in pre-randomize() advice (see EOF)
    constraint test02_scenarios {
        kind <= READS;
    }

    constraint writes {
        if (kind == WRITES) {
            length in {1:8};
            foreach (items, i) {
                items[i].kind == intel_mgmt_trans::WRITE;
            }
        }
    }
}
```

```

        items[i].data in {8'h00, 8'hFF, 8'hAA, 8'h55};
    }
}
constraint write_ro {
    if (kind == WRITE_R0) {
        length in {1:4};
        foreach (items, i) {
            items[i].kind == intel_mgmt_trans::WRITE;
            items[i].addr in {0:7};
        }
    }
}
constraint reads {
    if (kind == READS) {
        length in {1:8};
        foreach (items, i) {
            items[i].kind == intel_mgmt_trans::READ;
        }
    }
}
after task pre_randomize()
{
    // Pre-allocate enough scenario items
    // for the maximum-length scenario
    this.resize_items(8);
    void = constraint_mode(OFF, "valid_scenarios");
}
}

```